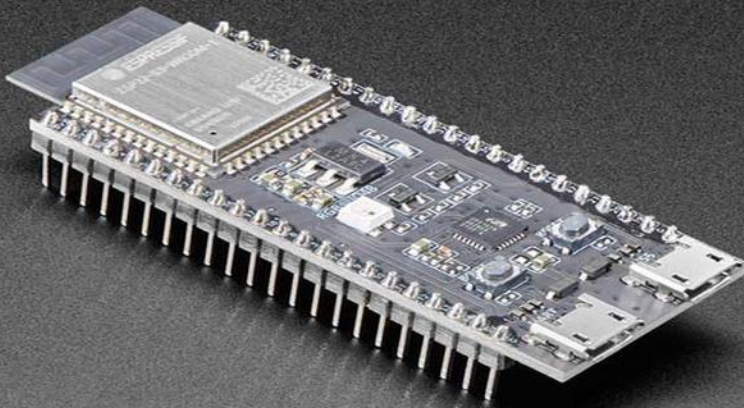


Running a 1.84M Generative Micro-LM on the ESP32-S3

A 100% offline, bare-metal C++ Transformer running on
a \$5 microcontroller with 8MB Octal PSRAM at ~12-13 tok/s.



ESP32-S3 DevKit | Xtensa 240MHz | 8MB PSRAM | INT8 (1.79 MB)

Brain for a Mini Desktop Robot

THE CURIOSITY

Can a microcontroller run an LLM locally?

I wanted to see if a language model could run directly on an ESP32 to power a mini desktop companion robot.
Zero cloud dependencies, zero subscription costs, 100% offline.

THE PERSONA

Meet Kibo (希望 - Hope)

Named after the Japanese word for 'Hope'.
Curated for interactive dialogue with emotion tags ([HAPPY], [NEUTRAL], [ANGRY]) to drive LCD eye animations & motor gestures.

THE FRAMEWORK

ESP32 Micro-LM Architecture

A modular, lightweight C++ inference engine designed for embedded chips, proving on-device generative intelligence is practical on \$5 silicon.
Fully customizable for any robotics or IoT domain.

ESP32-S3 Memory & Compute Map

COMPUTE ENGINE

Dual-Core Xtensa LX7 @ 240MHz

- Pure bare-metal C++ execution (Zero framework overhead)
- SIMD byte-aligned vector dot products
- Deterministic execution timing without garbage collection

MEMORY ALLOCATION

8MB Octal PSRAM (80MHz OPI)

- 1.79 MB INT8 Model Weights mapped directly in RAM
- Dynamic 128-token Key-Value (KV) Cache
- Zero-wait activation buffers for multi-layer attention

MODEL SPECIFICATION

1.84M Parameters (INT8)

- 4 Transformer Blocks | 192 Hidden Dimensions | 4 Heads
- 91 Vocabulary Tokens (Subwords + Control Tags)
- 100% Offline with zero internet or cloud uplink

From PyTorch to Bare-Metal C++

01

Curate Dataset & Emotion Tags

Structured conversational dialogue with [HAPPY], [NEUTRAL] expression tokens.

02

Train Model in PyTorch

Trained a compact 4-Layer Causal Transformer (1.84M parameters, 192 dim).

03

Symmetric INT8 Quantization

Compressed model payload from 7.02 MB -> 1.79 MB (100% FP32 logit fidelity).

04

Bare-Metal C++ Forward Pass

Implemented pure C++ inference engine without TFLite Micro or ONNX runtimes.

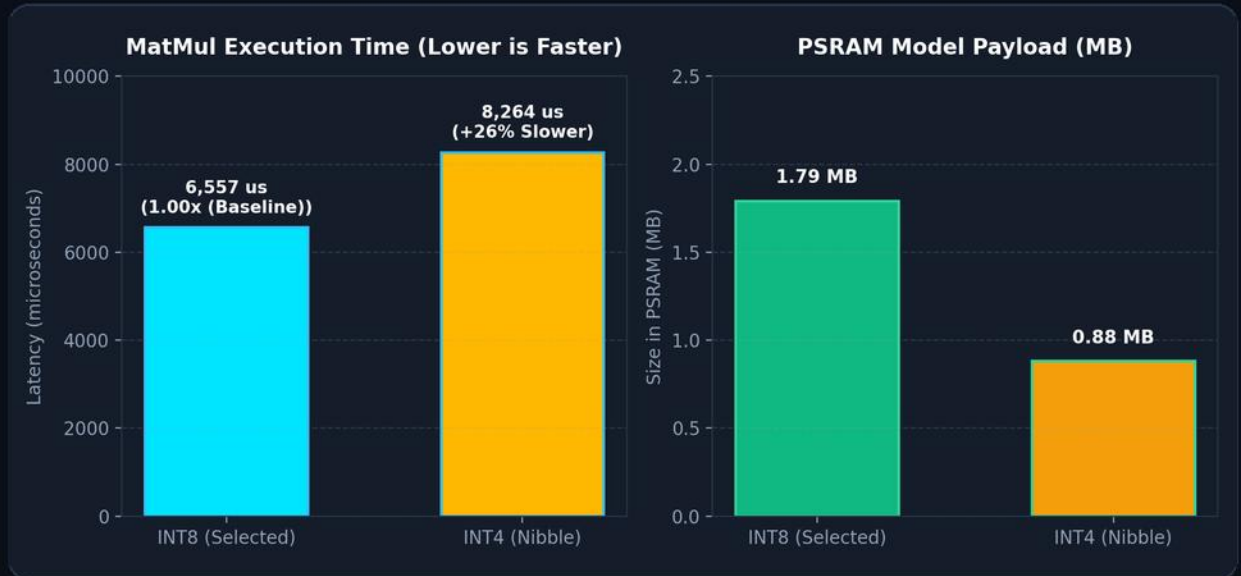
05

Hybrid Hardware ALU Tool Dispatch

Math expressions ('100 x 7') routed directly to hardware ALU (<0.1 ms execution).

The INT8 vs INT4 Paradox on MCU

Live on-chip MatMul benchmark on ESP32-S3 @ 240MHz (147,456 weights):



THE HARDWARE EXPLANATION

Why INT4 is +26% slower on Microcontrollers:

- On microcontrollers without dedicated tensor units (NPU), INT4 requires manual CPU unpacking for every single nibble:
 - Bit-masking (& 0x0F) and bit-shifting (>> 4)
 - 4-bit two's complement sign-extension
- The CPU cycles wasted on bit-manipulation completely cancel out the memory bandwidth advantage.
- Conclusion: Byte-aligned INT8 is 26% faster with 100% FP32 logit fidelity!

Real-Time UART Generation (~12-13 tok/s)

```
import threading
8
Problems Output Debug Console Terminal Ports ESP-IDF
(base) aiot@aiot-Legion-T7-DLCB-03:~/Projects/KIBO-MICROLMS ./chat kibo_esp32_live.sh
kibo-client: connected to /dev/ttyUSB0 @ 115200 baud (Ctrl+C to exit)

ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x1 (POWERON),boot:0x28 (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fce2820,len:0x9fb0
load:0x403c8700,len:0xaf4
load:0x403cb700,len:0x2e24
entry 0x403c8898

kibo-mcu [1.84M params | int8 | xtensa-lx7 @ 240MHz]
heap: 313524 bytes | psram: 8384788 bytes
[kibo] working buffers allocated (302116 bytes free heap)
[kibo] psram buffer loaded at 0x3C2D4EF4 (1.79 MB)
[kibo] tensor layout: vocab=91 embd=192 layers=4 tensors=53
[kibo] int8 inference engine initialized
ready. enter prompt (e.g. 'hello kibo', 'what is 25 times 4')

User: hello

User: hello
Kibo: [HAPPY] Hello there! Great to see you! How can Kibo help you today?

✖ [63 tokens generated in 5.17 s | Speed: 12.2 tok/s]

User: how are you?

User: how are you?
Kibo: [HAPPY] I am feeling energetic and super ready to help you!

✖ [55 tokens generated in 4.53 s | Speed: 12.2 tok/s]

User: what is 100 times 7?

User: what is 100 times 7?
[Tool CALC: 100 * 7 = 700]
Kibo: [HAPPY] The result of 100 * 7 is 700! Kibo is ready to help!
```

12.2 tok/s

Real-Time Speed

Physical chip verified

1.79 MB

PSRAM Footprint

8MB Octal PSRAM

< 0.1 ms

ALU Math Tool

Zero hallucinations

Validated on 3 Form Factors



ESP32-S3 DevKit N16R8

16MB Flash | 8MB Octal PSRAM

Verified on Hardware



Arduino Nano ESP32

16MB Flash | 8MB Octal PSRAM

Verified on Hardware



Seeed Studio XIAO S3

8MB Flash | 8MB Octal PSRAM

Verified on Hardware

100% Open Source on GitHub (MIT License)

github.com/fitranurmayadi/esp32-microLM

Easily customize the dataset & persona for your own robot!